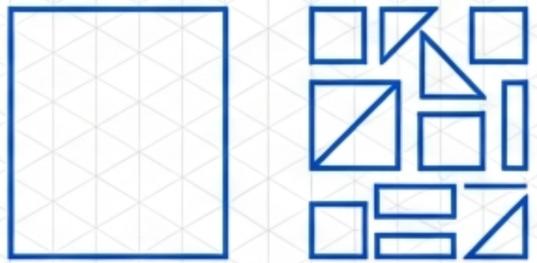
Tratamento Avançado de Exceções no ABAP OO

Definindo Classes, Interfaces T100 e Arquitetura de Erros Robustos

---

MANUAL TÉCNICO

Um guia prático para desenvolvedores sobre como transformar falhas técnicas em fluxos de controle gerenciáveis e comunicativos.



NotebookLM

A Filosofia do Erro: Além do SY-SUBRC

No ABAP OO, a exceção é um objeto de transporte, não apenas um sinal.

O ‘Dump’ (Falha Técnica)



- O programa aborta imediatamente
- O usuário vê uma tela técnica (ST22)
- Perda de dados em memória
- Sintoma: Código não preparado

O ‘Aviso de Negócio’ (Fluxo Controlado)



- O programa intercepta a falha
- O usuário recebe feedback traduzido
- Possibilidade de recuperação
- Sintoma: Arquitetura Resiliente

A exceção encapsula o estado (pilha, variáveis) e o transporta do ponto de falha até o manipulador.

NotebookLM

O Grande Divisor: Static Check vs. No Check

O contrato de tratamento entre emissor e receptor.

CX_STATIC_CHECK

Verificação Estática (Checked)



O Contrato: O compilador OBRIGA o tratamento.

Cenário: Regras de Negócio previsíveis (ex: Saldo Insuficiente).

Mentalidade: "O consumidor deve saber que isso pode acontecer e preparar uma contingência."

CX_NO_CHECK

Sem Verificação (Unchecked)



O Desastre: O compilador ignora.

Cenário: Falhas técnicas graves (ex: Memória Cheia, Pointer Nulo).

Mentalidade: "Recuperação improvável localmente. Deixar subir para um Global Handler."

NotebookLM

O Meio-Termo e a Matriz de Decisão

CX_DYNAMIC_CHECK

Definição: Verificação Híbrida em Runtime. Usado em frameworks genéricos onde a exceção só é conhecida durante a execução.

Matriz de Decisão (Cheat Sheet)

Tipo	Classificação	Obrigatório Tratar?	Exemplo Típico
CX_STATIC_CHECK	Checked	SIM (Compilador cobra)	Regra de Negócio (Saldo Baixo)
CX_NO_CHECK	Unchecked	NÃO	Erro Técnico (Dump/Memória)
CX_DYNAMIC_CHECK	Híbrida	NÃO (Check em Runtime)	Frameworks Dinâmicos

Regra de Ouro: Se o consumidor do método *pode* e *deve* fazer algo a respeito do erro, use Static Check.

NotebookLM

Falando a Língua do Usuário (Interface T100)

O Problema:

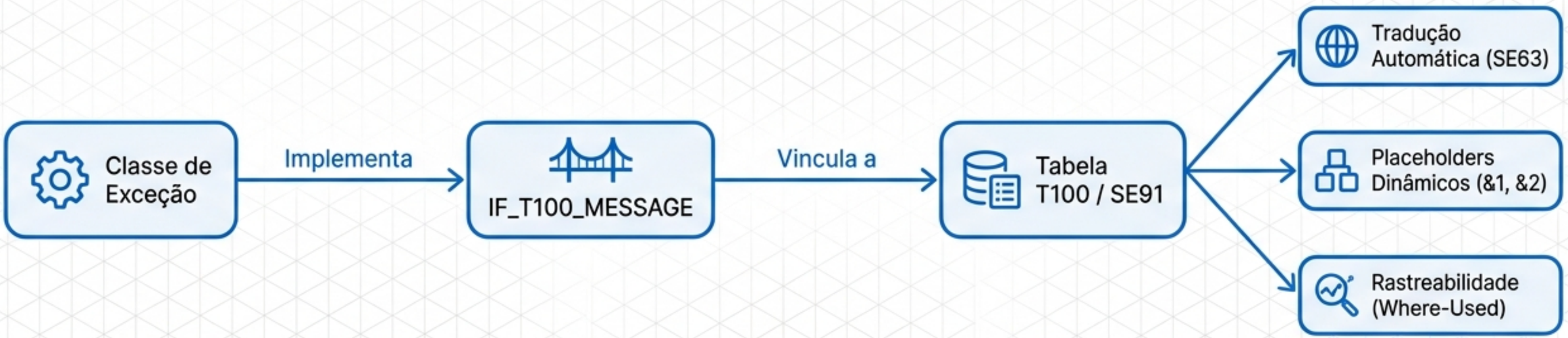
Exceções genéricas ou textos ‘hardcoded’ no código.

Resultado:

Impossível traduzir, difícil de rastrear.

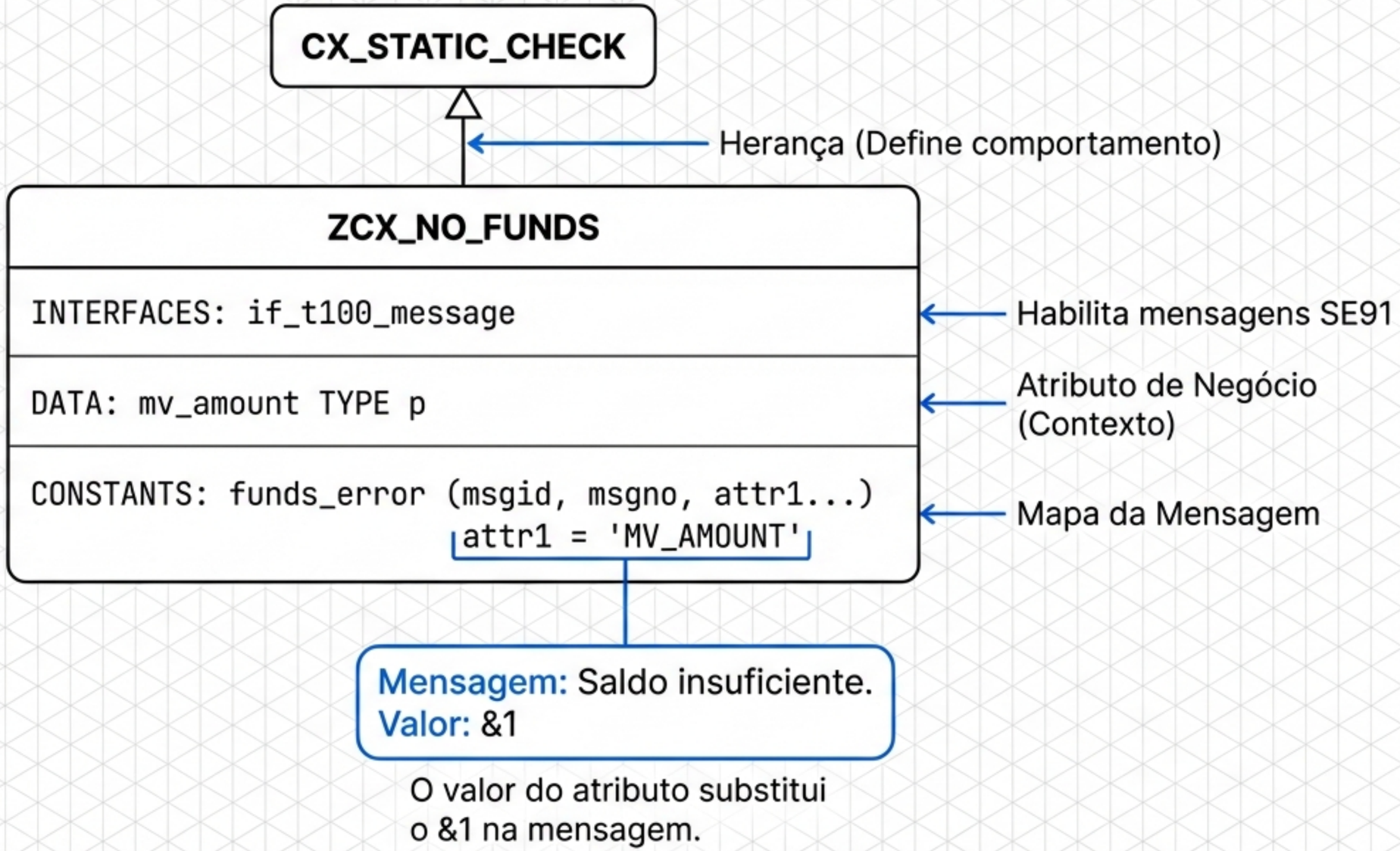
IF_T100_MESSAGE

A ponte entre o código ABAP e a transação SE91.



NotebookLM

Anatomia de uma Classe de Exceção Rica



NotebookLM

Blueprint: Definindo a Classe (Código)

```
CLASS zcx_no_funds DEFINITION INHERITING FROM cx_static_check.
  PUBLIC SECTION.
    INTERFACES if_t100_message.

    DATA mv_amount TYPE p LENGTH 15 DECIMALS 2.

    CONSTANTS: BEGIN OF funds_error,
                 msgid TYPE symsgid VALUE 'ZFINANCE_MSG',
                 msgno TYPE symsgno VALUE '001',
                 attr1 TYPE scx_attrname VALUE 'MV_AMOUNT',
               END OF funds_error.

    METHODS constructor IMPORTING textid LIKE if_t100_message=>t100key OPTIONAL
                                  previous LIKE previous OPTIONAL
                                  amount TYPE p OPTIONAL.
ENDCLASS.
```

Mapeamento: Liga o placeholder &1 ao atributo MV_AMOUNT.

Construtor recebe TEXTID (qual mensagem) e PREVIOUS (cadeia de erros).

NotebookLM

A Implementação do Construtor

```
METHOD constructor.

  CALL METHOD super->constructor
    EXPORTING previous = previous.

  me->mv_amount = amount.

  CLEAR me->textid.
  IF textid IS INITIAL.
    if_t100_message~t100key = if_t100_message=>default_textid.
  ELSE.
    if_t100_message~t100key = textid.
  ENDIF.

ENDMETHOD.
```

1 **Vital**: Gerencia a pilha de erros e o parâmetro `PREVIOUS`.

2 **Persistência**: Salva o contexto para uso futuro.

3 **Seleção**: Define qual mensagem da SE91 será exibida.

NotebookLM

O Disparo: RAISE EXCEPTION

Interrompendo o fluxo e transferindo o controle.

```
  METHOD zcx_no_funds
    IF iv_amount = iv_amount,
      mv_balance = mv_balance.
    ENDIF.
    RAISE EXCEPTION TYPE zcx_no_funds
      EXPORTING
        textid  = zcx_no_funds=>funds_error
        amount  = iv_amount.
  ENDIF.
ENDMETHOD.
```

Instancia a classe, preenche 'amount' com o valor do erro e aborta o método atual.



Saída Imediata

NotebookLM

A Defesa: O Bloco TRY...CATCH

```
TRY.
    lo_account->debit_account( 500 ).

  CATCH zcx_no_funds INTO DATA(lx_error).
    DATA(lv_msg) = lx_error->get_text( ),
    out->write( |Erro: { lv_msg } - Valor: { lx_error->mv_amount }| ).

  CATCH cx_root INTO DATA(lx_generic).
    out->write( 'Erro técnico grave e desconhecido.' ).

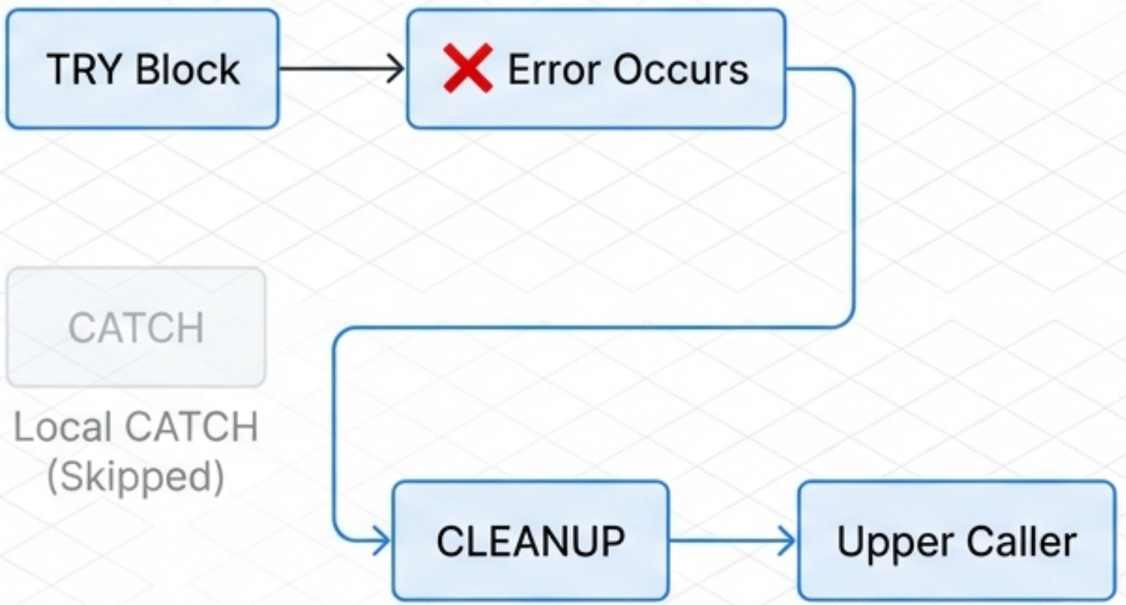
ENDTRY.
```

Tratamento Específico: Acesso aos dados de negócio (mv_amount) para recuperação inteligente.

Polimorfismo: Rede de segurança (Catch-all) para erros técnicos não previstos.

NotebookLM

Housekeeping: O Bloco CLEANUP




```
CLEANUP.
  out->write( 'Limpando recursos críticos
antes de abortar...' ).
  " Ex: fechar arquivos, liberar bloqueios
```

- **Conceito:** Executado quando o erro ocorre mas **NÃO** é capturado localmente.
- **Uso:** Fechar conexões ou limpar memória antes de perder o controle do escopo.

NotebookLM

Modernização: Sintaxe ABAP 7.50+

Interface IF_T100_DYN_MSG

Dispare exceções com a simplicidade do comando MESSAGE, gerando objetos automaticamente.

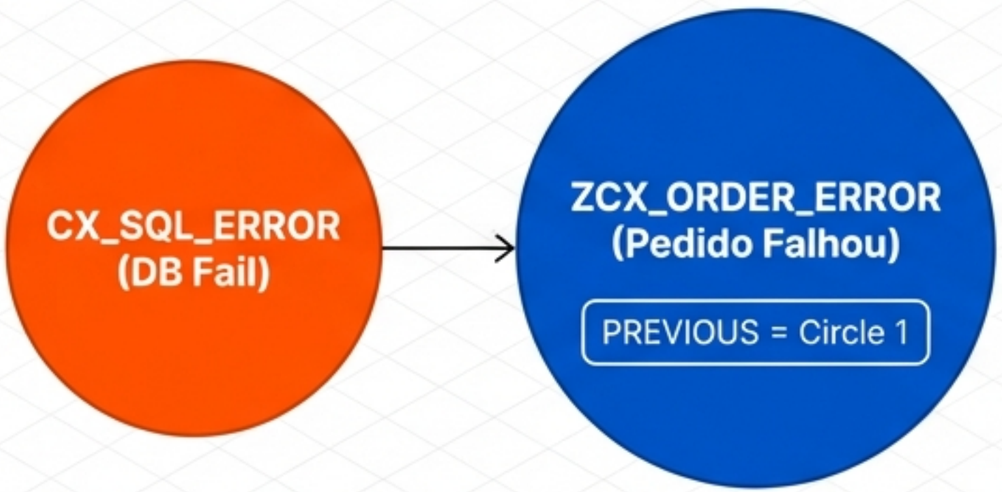
```
RAISE EXCEPTION TYPE zcx_generic_error
  MESSAGE ID 'ZMSG'
  TYPE 'E'
  NUMBER '001'
  WITH 'Parametro1' 'Parametro2'.
```

- Menos código boilerplate
- Preenchimento automático da T100
- Substituição dinâmica de variáveis

NotebookLM

Técnicas Avançadas: Encadeamento (PREVIOUS)

Um erro técnico (SQL) ocorre, mas precisamos relatar um erro de negócio.




```
CATCH cx_sql_error INTO DATA(lx_sql).
  RAISE EXCEPTION TYPE zcx_order_error
    EXPORTING
      previous = lx_sql. " O Rastro de Migalhas
```

O parâmetro PREVIOUS mantém a causa raiz acessível para debug, mesmo após a transformação do erro.

NotebookLM

Quiz de Fixação

1. O que acontece se eu não tratar uma CX_STATIC_CHECK?

Erro de sintaxe. O compilador impede a ativação para garantir o contrato.

---

2. Qual a função da interface IF_T100_MESSAGE?

Vincular a exceção à tabela T100 (SE91), permitindo tradução e **placeholders**.

---

3. Qual a diferença entre CATCH e CLEANUP?

CATCH captura e trata o erro. CLEANUP executa apenas se o erro NÃO for capturado localmente (limpeza).

NotebookLM

Resumo Estratégico & Glossário

Glossário

Exception Class: Encapsula o erro e dados.

RAISE: Interrompe o fluxo imediatamente.

TRY/CATCH: Captura, trata e recupera.

CLEANUP: Limpa recursos se o erro subir.

IF_T100_MESSAGE: Interface de Mensagens.

"O tratamento de exceções não é sobre evitar erros, é sobre gerenciar o imprevisto com elegância, garantindo que o software falhe de forma segura, limpa e informativa."

NotebookLM